

```
In [2]: # IMPORT LIB
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

```
In [5]: # conntion of sql to database
conn = sqlite3.connect("customer_churn.db")

# retrieve table names
sql_query = "SELECT name FROM sqlite_master WHERE type='table'"
tables = pd.read_sql(sql_query, conn)

# Load each table into globals
for table_name in tables["name"]:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created dataframe : df_{table_name}")

# close the connection only after all queries complete
conn.close()
```

Created dataframe : df_db_customer
Created dataframe : df_db_subscription
Created dataframe : df_db_support

```
In [4]: # print table name and column names
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTables Name : {table_name}")

    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns: ")
    print(columns['name'].tolist())

    #Close connection
conn.close()
```

Tables Name : db_customer

Columns:

['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Tables Name : db_subscription

Columns:

['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score']

Tables Name : db_support

Columns:

['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']

In [87]: *# DATA CLEANING*

```
# for getting n top result from tables  
df_db_customer.head(5)
```

Out[87]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> </tbody> <tbody></tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
------------	--------	-------	-------------	------	---------------------

0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
------------	--------	-------	-----------	------	---------------------

0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
------------	--------	-------	-------	--------	---------------------

0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
------------	-------	-------	----------	------	---------------------

0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
------------	------	-------	-------	--------	---------------------

In []:

In [29]: *# for getting bottam n result from tables*
df_db_customer.tail(5)

```
Out[29]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead>
<tbody> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody>
<tbody></tbody>
```

0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

```
In [ ]:
```

```
In [30]: # for getting info about tables
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   customerid  21 non-null    str
1   name        21 non-null    str
2   country     18 non-null    str
3   state       21 non-null    str
4   gender      21 non-null    str
5   dob         21 non-null    str
6   interests   4 non-null     str
7   pincode     0 non-null     object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

```
In [ ]:
```

```
In [31]: # for rename the columns name (.rename) use {} bracket and inplace is act like palc
df_db_customer.rename(columns = {'name': 'cust_name'}, inplace=True )
```

```
In [ ]:
```

```
In [30]: # for drop the columns (.drop) use [] bracket
# df_db_customer.drop(df_db_customer.columns[-2:], axis=1)
# df_db_customer.drop(df_db_customer.columns[6:], axis=1)

df_db_customer.drop(columns=['interests', 'pincode'], inplace=True)
```

```
In [34]: df_db_customer.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   customerid  21 non-null    str
 1   cust_name   21 non-null    str
 2   country     18 non-null    str
 3   state       21 non-null    str
 4   gender      21 non-null    str
 5   dob         21 non-null    str
dtypes: str(6)
memory usage: 1.1 KB

```

In []:

```

In [35]: # for changing the datatype of columns DOB (str to datetime ) by using pandas
df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])

```

```

In [36]: df_db_customer.info()

```

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   customerid  21 non-null    str
 1   cust_name   21 non-null    str
 2   country     18 non-null    str
 3   state       21 non-null    str
 4   gender      21 non-null    str
 5   dob         21 non-null    datetime64[us]
dtypes: datetime64[us](1), str(5)
memory usage: 1.1 KB

```

In []:

```

In [38]: # data standardization - gender
df_db_customer['gender'].unique()

```

```

Out[38]: <StringArray>
['Male', 'Female', 'Women', 'Men']
Length: 4, dtype: str

```

```

In [40]: # for relacing Men to Male and Women to Female (.replace)
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men': 'Male', 'Women'

```

```

In [41]: df_db_customer['gender'].unique()

```

```

Out[41]: <StringArray>
['Male', 'Female']
Length: 2, dtype: str

```

In []:

```
In [43]: # for fixing missing value (null) or identify no.(null) value (.isna)
# for calculating no. of null value (.sum)

df_db_customer['country'].isna().sum()
```

```
Out[43]: np.int64(3)
```

```
In [46]: df_db_customer[df_db_customer['country'].isna()]
```

```
Out[46]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> </thead> <tbody> <th>5</th> <th>8</th>
<th>12</th> </tbody> <tbody></tbody>
```

0013-MHZWF	durga	NaN	Delhi	Female	1988-12-10 00:00:00
------------	-------	-----	-------	--------	---------------------

0015-UOCOJ	maya	NaN	Kathmandu	Female	1985-07-07 00:00:00
------------	------	-----	-----------	--------	---------------------

0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00
------------	--------	-----	-----------	--------	---------------------

```
In [ ]:
```

```
In [48]: # country and state - unique value pair
# Creating state → country map from non-null rows

state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('state'

# Fill the missing country using State
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['state'
```

```
In [49]: df_db_customer[df_db_customer['country'].isna()] # no null value in country col
```

```
Out[49]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> </thead> <tbody> </tbody> <tbody></tbody>
```

```
In [ ]:
```

```
In [51]: df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   customerid  21 non-null    str
1   name        21 non-null    str
2   country     21 non-null    str
3   state       21 non-null    str
4   gender      21 non-null    str
5   dob         21 non-null    str
dtypes: str(6)
memory usage: 1.1 KB
```

```
In [ ]:
```

```
In [54]: # subscription table

df_db_subscription.head()
```

```
Out[54]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th>
<th>renewal_date</th> <th>plan_type</th> <th>contract_type</th>
<th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th>
<th>cltv</th> <th>churn_score</th> </thead> <tbody> <th>0</th> <th>1</th>
<th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaN	NaN	13.99	627	12
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaN	NaN	6.99	210	34
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaN	NaN	22.99	1725	8
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88

```
In [ ]:
```

```
In [55]: df_db_subscription.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    str
1   subscription_start_date 21 non-null    str
2   subscription_type      21 non-null    str
3   renewal_date          21 non-null    str
4   plan_type              21 non-null    str
5   contract_type          21 non-null    str
6   cancellation_date       6 non-null     str
7   cancellation_reason     6 non-null     str
8   monthly_charges        21 non-null    float64
9   cltv                   21 non-null    int64
10  churn_score             21 non-null    int64
dtypes: float64(1), int64(2), str(8)
memory usage: 1.9 KB

```

In []:

```

In [58]: # change the datatype of columns (subscription_start_date , renewal_date , cancella
date_columns = ['subscription_start_date' , 'renewal_date' , 'cancellation_date']
df_db_subscription[date_columns] = df_db_subscription[date_columns].apply(pd.to_dat

```

In [59]: df_db_subscription.info()

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    str
1   subscription_start_date 21 non-null    datetime64[us]
2   subscription_type      21 non-null    str
3   renewal_date          21 non-null    datetime64[us]
4   plan_type              21 non-null    str
5   contract_type          21 non-null    str
6   cancellation_date       6 non-null     datetime64[us]
7   cancellation_reason     6 non-null     str
8   monthly_charges        21 non-null    float64
9   cltv                   21 non-null    int64
10  churn_score             21 non-null    int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB

```

In []:

```

In [60]: # support table
df_db_support.head()

```

```
Out[60]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th>
<th>csat_score</th> <th>col_1</th> <th>comment</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN

```
In [61]: df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customerid      9 non-null     str
1   complaint_date  9 non-null     str
2   escalations     9 non-null     str
3   csat_score      9 non-null     int64
4   col_1           0 non-null     object
5   comment         4 non-null     str
dtypes: int64(1), object(1), str(4)
memory usage: 564.0+ bytes
```

```
In [ ]:
```

```
In [62]: #drop
df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

```
In [ ]:
```

```
In [63]: # change data type to date - complaint_date

df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])

df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   customerid            9 non-null     str
1   complaint_date        9 non-null     datetime64[us]
2   escalations           9 non-null     str
3   csat_score            9 non-null     int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 420.0 bytes
```

```
In [ ]:
```

```
In [74]: # FEATURE ENGINEERING & DATA ANALYSIS
```

```
In [69]: df_db_subscription.columns
```

```
Out[69]: Index(['customerid', 'subscription_start_date', 'subscription_type',
               'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
               'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score'],
              dtype='str')
```

```
In [73]: # create columns using exiting columns
#customer is churn if cancelation date in not null

#df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date']

df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date']
df_db_subscription.head()
```

```
Out[73]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th>
<th>renewal_date</th> <th>plan_type</th> <th>contract_type</th>
<th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th>
<th>cltv</th> <th>churn_score</th> <th>churn_flag</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	12	0
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	34	0
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	8	0
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88	1

In []:

In [6]: *# MERGE DATAFRAMES - JOINS*

```
# first fix support table duplicates then merge  
# Note: while merging df's always check the shape before and after  
df = (df_db_subscription  
      .merge(df_db_customer, on = 'customerid', how= 'left')  
      .merge(df_db_support, on = 'customerid', how= 'left') )
```

In [7]: df_db_subscription.shape

Out[7]: (21, 11)

In [8]: df.shape

Out[8]: (23, 23)

```
In [9]: print('df_db_subscription unique value:', df_db_subscription['customerid'].nunique()  
print('df_db_customer unique value:', df_db_customer['customerid'].nunique())  
print('df_db_support unique value:', df_db_support['customerid'].nunique())  
print('df_db_support all value:', df_db_support['customerid'].size)
```

```
df_db_subscription unique value: 21  
df_db_customer unique value: 21  
df_db_support unique value: 7  
df_db_support all value: 9
```

In [10]: df_db_support

Out[10]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th>
<th>csat_score</th> <th>col_1</th> <th>comment</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th>
<th>8</th> </tbody> <tbody></tbody>

0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN
0017-IUDMW	2024-04-10 00:00:00	Y	25	None	NaN
0019-EFAEP	2024-09-27 00:00:00	Y	30	None	NaN
0022-TCJCI	2024-09-13 00:00:00	Y	10	None	NaN
0022-TCJCI	2024-09-14 00:00:00	N	90	None	received refund

```

In [11]: df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid']

In [12]: df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('customerid')

In [13]: df_db_support['customerid'].size

Out[13]: 7

In [14]: # merge df
df = (df_db_subscription
      .merge(df_db_customer, on = 'customerid', how= 'left')
      .merge(df_db_support, on = 'customerid', how= 'left') )

In [16]: df.shape

Out[16]: (21, 24)

In [19]: # export dataframe to csv file

df.to_csv('exported_churn_data.csv', index=False)

In [ ]:

In [20]: # DATAT ANALYSIS

In [31]: # Print all columns
print(df.columns.tolist())

# Or search specifically for columns containing 'churn'
print([col for col in df.columns if "churn" in col.lower()])

['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment', 'complaint_count', 'churn_flag']
['churn_score', 'churn_flag']

In [25]: df.head(2)

```

```
Out[25]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th>
<th>renewal_date</th> <th>plan_type</th> <th>contract_type</th>
<th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th>
<th>cltv</th> <th>...</th> <th>gender</th> <th>dob</th> <th>interests</th>
<th>pincode</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th>
<th>col_1</th> <th>comment</th> <th>complaint_count</th> </thead> <tbody>
<th>0</th> <th>1</th> </tbody> <tbody></tbody>
```

0002-ORFBO	2021-03-15	Referral	2025-03-15	Standard	Annual	NaN	NaN	13.99	627	...	Male
------------	------------	----------	------------	----------	--------	-----	-----	-------	-----	-----	------

0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	Male
------------	------------	------	------------	---------	--------	------------	------------------------	-------	------	-----	------

<p>2 rows × 24 columns</p>



```
In [26]: # 1. Churn Rate
# Create the binary churn_flag (1 if cancelled/churned, 0 if active)
df["churn_flag"] = df["cancellation_date"].notna().astype(int)

# Calculate the actual churn rate
churn_rate = df["churn_flag"].mean() * 100
print(f"Overall Churn Rate = {churn_rate:.2f}%")

# See exact counts of Active vs Churned customers
print("\nCustomer Status Counts:")
print(
    df["churn_flag"].value_counts().rename(index={0: "Active", 1: "Churned"})
)
```

Overall Churn Rate = 28.57%

Customer Status Counts:

churn_flag	
Active	15
Churned	6

Name: count, dtype: int64

```
In [28]: # 2. Retention Rate
retention_rate = 100 - churn_rate
print("Retention Rate = ", round(retention_rate,2), "%")
```

Retention Rate = 71.43 %

```
In [29]: # 3. Churn by Plan type
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).res
print(churn_by_plan)
```

	plan_type	churn_rate_pct
0	Basic	60.00
1	Premium	14.29
2	Standard	22.22

In []:

In [32]: *# 4.a Churn by state + sum(revenue) & count of users*

```
# Ensure churn_flag is created
df["churn_flag"] = df["cancellation_date"].notna().astype(int)

# Group by state and aggregate metrics
state_summary = (
    df.groupby("state")
    .agg(
        total_users=("customerid", "nunique"),
        churned_users=("churn_flag", "sum"),
        churn_rate_pct=("churn_flag", lambda x: round(x.mean() * 100, 2)),
        total_monthly_revenue=("monthly_charges", "sum"),
        total_cltv=("cltv", "sum"),
    )
    .sort_values(by="total_users", ascending=False)
    .reset_index()
)

# Format revenue for clean display
state_summary["total_monthly_revenue"] = state_summary[
    "total_monthly_revenue"
].round(2)
state_summary["total_cltv"] = state_summary["total_cltv"].round(2)

print(state_summary)
```

	state	total_users	churned_users	churn_rate_pct	\
0	Delhi	4	1	25.00	
1	Meghalaya	3	2	66.67	
2	Maharashtra	3	0	0.00	
3	Kathmandu	2	0	0.00	
4	Karnataka	2	2	100.00	
5	Rajasthan	2	0	0.00	
6	Telangana	2	1	50.00	
7	Uttar Pradesh	2	0	0.00	
8	Nagaland	1	0	0.00	

	total_monthly_revenue	total_cltv
0	52.96	1765
1	42.97	940
2	50.97	3027
3	20.98	575
4	20.98	1420
5	36.98	2680
6	30.98	1022
7	115.98	4140
8	22.99	1725

In [33]: # 4.b Churn by subscription type + sum(revenue) & count of users

```
# Create a simple churn column (1 = Churned, 0 = Active)
df["churn"] = df["cancellation_date"].notna().astype(int)

# Group by subscription_type and calculate the numbers
result = (
    df.groupby("subscription_type")
    .agg(
        {
            "customerid": "count", # Count of users
            "monthly_charges": "sum", # Sum of revenue
            "churn": "sum", # Count of churned users
        }
    )
    .reset_index()
)

# Rename columns so they are easy to read
result.columns = [
    "Subscription Type",
    "Total Users",
    "Total Revenue",
    "Churned Users",
]

print(result)
```

	Subscription Type	Total Users	Total Revenue	Churned Users
0	Organic	9	145.91	0
1	Paid	6	174.94	1
2	Refferal	6	74.94	5

In []:

In [34]: # 5. ARPU - Avg Revenue per user

```
arpu = df['monthly_charges'].mean()
print('ARPU = ', round(arpu,2))
```

ARPU = 18.85

In []:

In [36]: # 6. Avg Customer Tenure

```
# count of days users has used our service : cancellation date else current date
df["subscription_start_date"] = pd.to_datetime(
    df["subscription_start_date"], errors="coerce"
)
df["cancellation_date"] = pd.to_datetime(
    df["cancellation_date"], errors="coerce"
)

# Get today's date
today = pd.Timestamp.today()

# Calculate tenure in days
```

```
df["tenure_days"] = np.where(
    df["cancellation_date"].notna(),
    (df["cancellation_date"] - df["subscription_start_date"]).dt.days,
    (today - df["subscription_start_date"]).dt.days,
)

# Calculate and print average tenure
avg_tenure = df["tenure_days"].mean()
print("Avg Tenure (Days) = ", round(avg_tenure, 2))
```

Avg Tenure (Days) = 1518.71

In []:

```
In [37]: # 7. Revenue at risk - revenue lost from churned users
revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
print("Revenue at Risk (Rs 'K') =", revenue_at_risk)
```

Revenue at Risk (Rs 'K') = 73.94

In []:

```
In [38]: # 8. Escalation Rate
escalation_rate = (df['escalations']=='Y').mean()*100
print("Escalation Rate = ", round(escalation_rate, 2), "%")
```

Escalation Rate = 19.05 %

In []:

```
In [39]: # 9. Avg Complaint Per User
avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg Compliants Per User = ", round(avg_complaints, 2))
```

Avg Compliants Per User = 0.43

In []:

```
In [40]: # 10. Correlation Escalation vs Churn
df['escalations'] = np.where(df['escalations'] == 'Y', 1, 0) # encoding string to i
corr_df = df[['escalations', 'churn_flag']].dropna()

correlation = corr_df['escalations'].corr(df['churn_flag'])
print("Correlation between escalation vs churn is = ", round(correlation,2))
```

Correlation between escalation vs churn is = 0.77

In []:

```
In [41]: # 11. Create a column using existing col - Churn risk
conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50) & (df['churn_score'] < 70),
    (df['churn_score'] >= 70)
]

choices = ['low', 'med', 'high']
```

```
df['churn_risk'] = np.select(conditions, choices, default='unkown')
```

```
In [42]: df['churn_risk']
```

```
Out[42]: 0      low
          1      high
          2      low
          3      low
          4      high
          5      low
          6      high
          7      low
          8      low
          9      low
         10      low
         11      high
         12      low
         13      high
         14      low
         15      med
         16      med
         17      low
         18      high
         19      low
         20      low
          Name: churn_risk, dtype: str
```

```
In [ ]:
```

```
In [43]: # VISUALIZATION USING MATPLOTLIB
```

```
In [44]: df_visual = df.copy()
```

```
In [46]: df_visual.columns
```

```
Out[46]: Index(['customerid', 'subscription_start_date', 'subscription_type',
               'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
               'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'name',
               'country', 'state', 'gender', 'dob', 'interests', 'pincode',
               'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment',
               'complaint_count', 'churn_flag', 'churn', 'tenure_days', 'churn_risk'],
              dtype='str')
```

```
In [ ]:
```

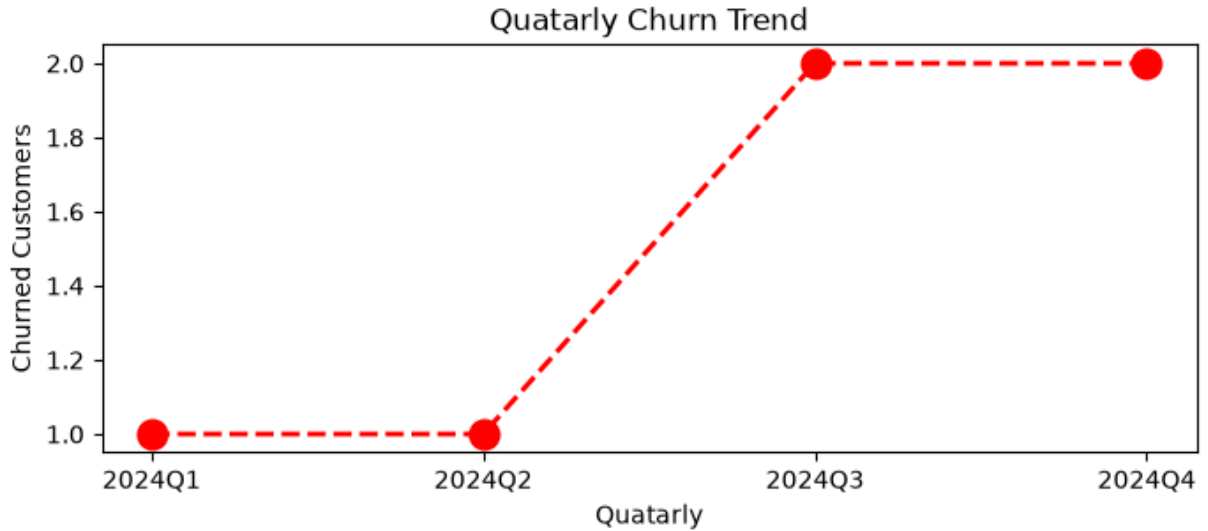
```
In [56]: # Quatarly Churn Trend (Time Series KPI)
```

```
df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('Q')

churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month')

plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values, color='red', marker='o')
```

```
plt.title('Quatarly Churn Trend')
plt.xlabel('Quatarly')
plt.ylabel('Churned Customers')
plt.show()
```



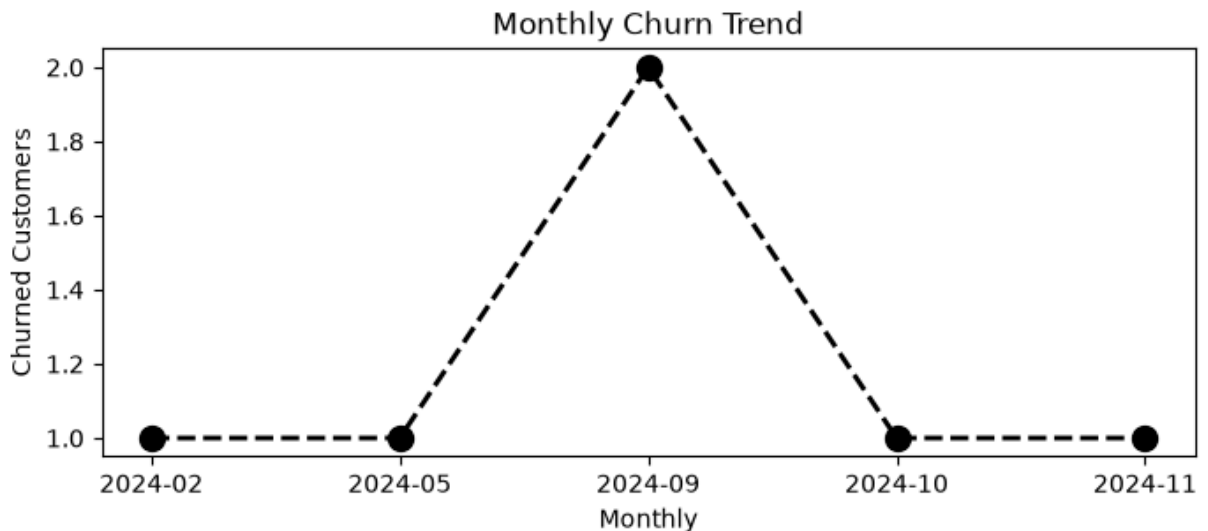
```
In [59]: # Monthly Churn Trend (Time Series KPI)

df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')

churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month')

plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values, color='black', marker='o')

plt.title('Monthly Churn Trend')
plt.xlabel('Monthly')
plt.ylabel('Churned Customers')
plt.show()
```

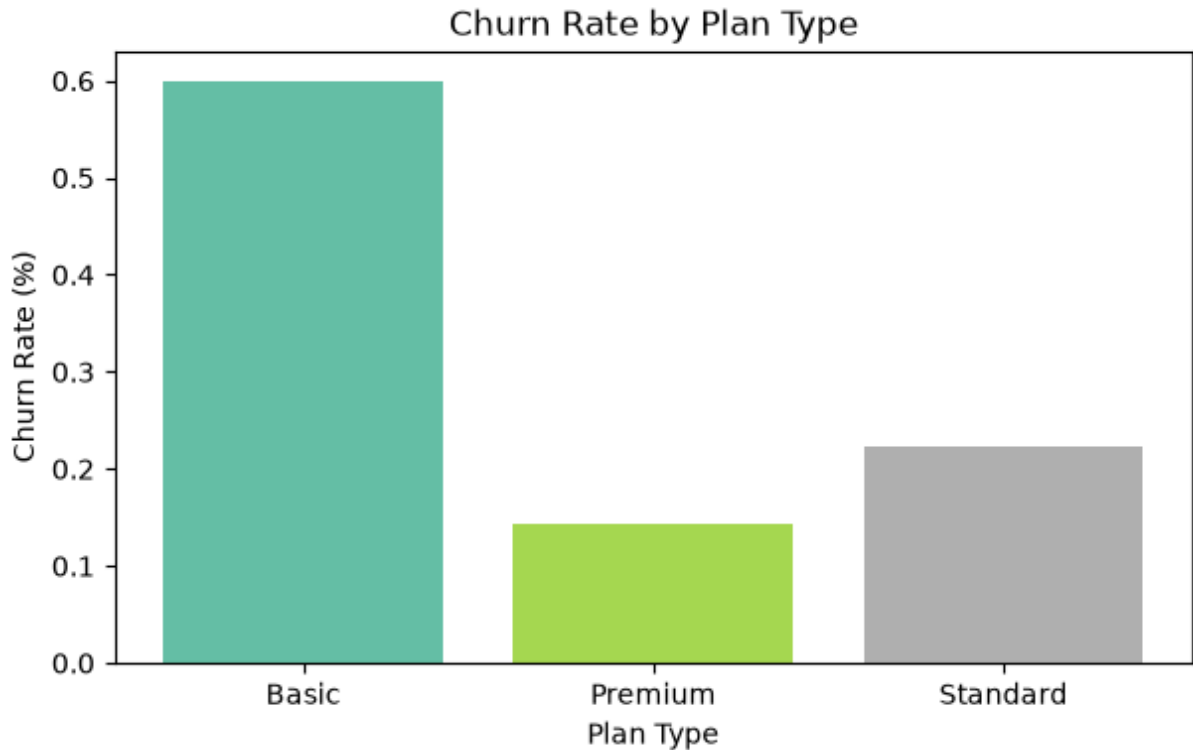


```
In [60]: #Churn Rate by Plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()
```

```
# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(7,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Plan Type')
plt.xlabel('Plan Type')
plt.ylabel('Churn Rate (%)')
plt.show()
```



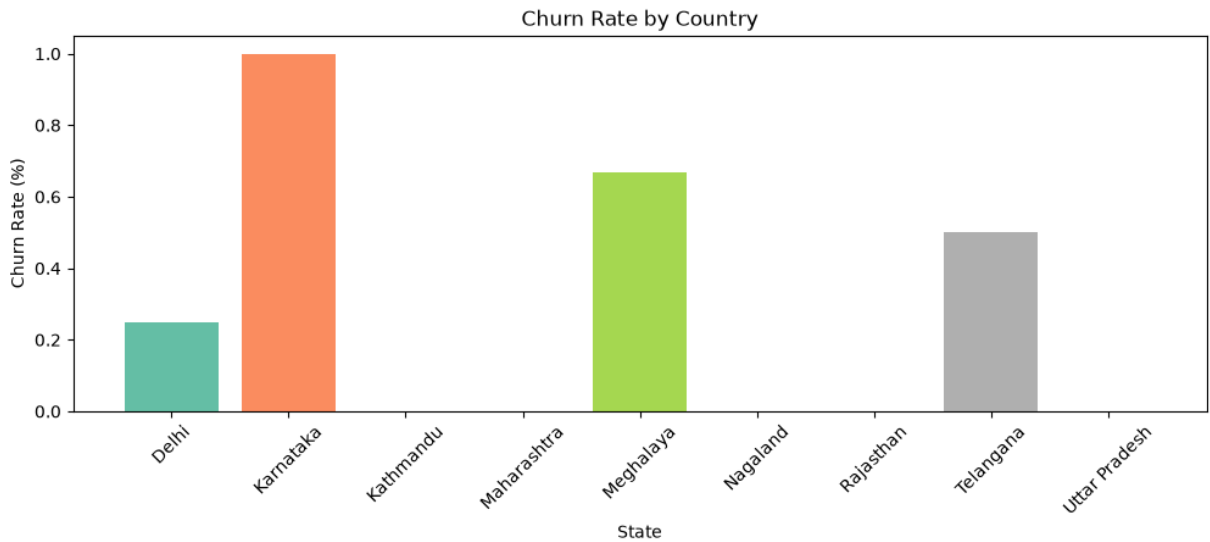
In []:

```
In [63]: # Churn by States
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(12,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Country')
plt.xlabel('State')
plt.ylabel('Churn Rate (%)')
plt.xticks(rotation=45)
plt.show()
```



In []:

In [64]: `# VISUALIZATION USING SEABORN`

In [65]: `# Heatmap (Correlation Matrix)`

In [66]: `# encoding - convert str to numeric so that we can find corr between features`
`df_visual.columns`

Out[66]: Index(['customerid', 'subscription_start_date', 'subscription_type',
'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'name',
'country', 'state', 'gender', 'dob', 'interests', 'pincode',
'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment',
'complaint_count', 'churn_flag', 'churn', 'tenure_days', 'churn_risk',
'cancellation_month'],
dtype='str')

In [67]: `df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk',`

Out[67]: `<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody`
`tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>`
`</th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th>`
`<th>churn_flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody>`
`<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>`
`</tbody>`

Standard	Annual	12	0	low	0
----------	--------	----	---	-----	---

Premium	Annual	91	1	high	1
---------	--------	----	---	------	---

Basic	Monthly	34	0	low	0
-------	---------	----	---	-----	---

Premium	Annual	8	0	low	0
---------	--------	---	---	-----	---

Standard	Monthly	88	1	high	1
----------	---------	----	---	------	---

```
In [68]: # remove warnings
```

```
import warnings
warnings.filterwarnings("ignore")
```

```
In [69]: # incorrect method of encoding - as numbers are not assigned based on priority
```

```
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag',
```

```
                        'churn_risk']]
categorical_cols = ['plan_type', 'contract_type', 'churn_risk']
```

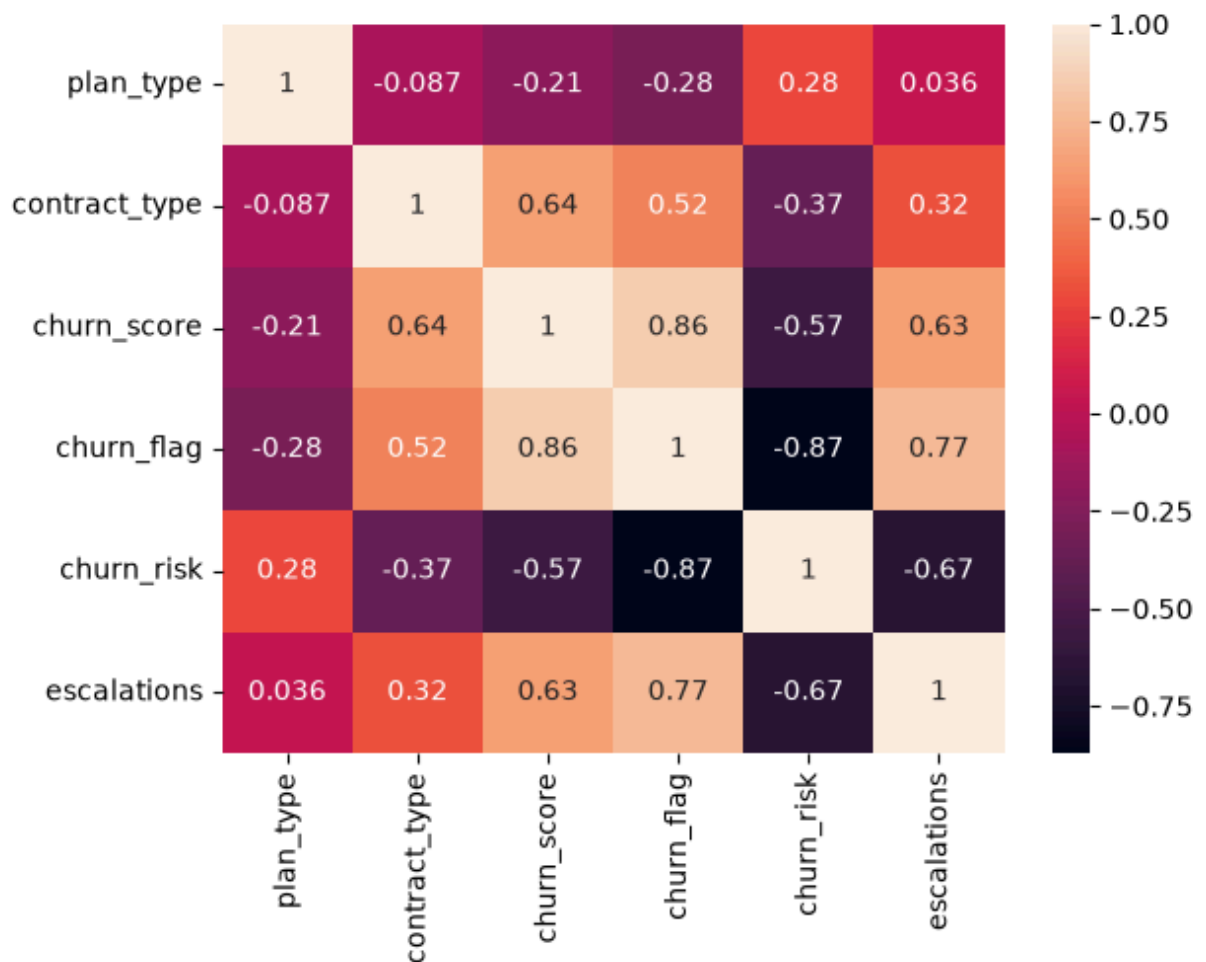
```
for col in categorical_cols:
```

```
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

```
In [70]: # Heatmap (correlation matrix)
```

```
sns.heatmap(df_encoded.corr(), annot=True)
```

```
Out[70]: <Axes: >
```



```
In [71]: print('plan_type :', df_visual['plan_type'].unique())
```

```
print('contract_type :', df_visual['contract_type'].unique())
```

```
print('churn_risk :', df_visual['churn_risk'].unique())
```

```

plan_type : <StringArray>
['Standard', 'Premium', 'Basic']
Length: 3, dtype: str
contract_type : <StringArray>
['Annual', 'Monthly']
Length: 2, dtype: str
churn_risk : <StringArray>
['low', 'high', 'med']
Length: 3, dtype: str

```

```

In [72]: # Correct method of encoding - based on priority
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag',

order_mappings = {
    'plan_type' : ['Basic', 'Standard', 'Premium'],
    'contract_type' : ['Monthly', 'Annual'],
    'churn_risk': ['low', 'med', 'high']
}

for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories

```

```

In [73]: df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk',

```

```

Out[73]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th>
<th>churn_flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>

```

Standard	Annual	12	0	low	0
Premium	Annual	91	1	high	1
Basic	Monthly	34	0	low	0
Premium	Annual	8	0	low	0
Standard	Monthly	88	1	high	1

```

In [74]: df_encoded.head()

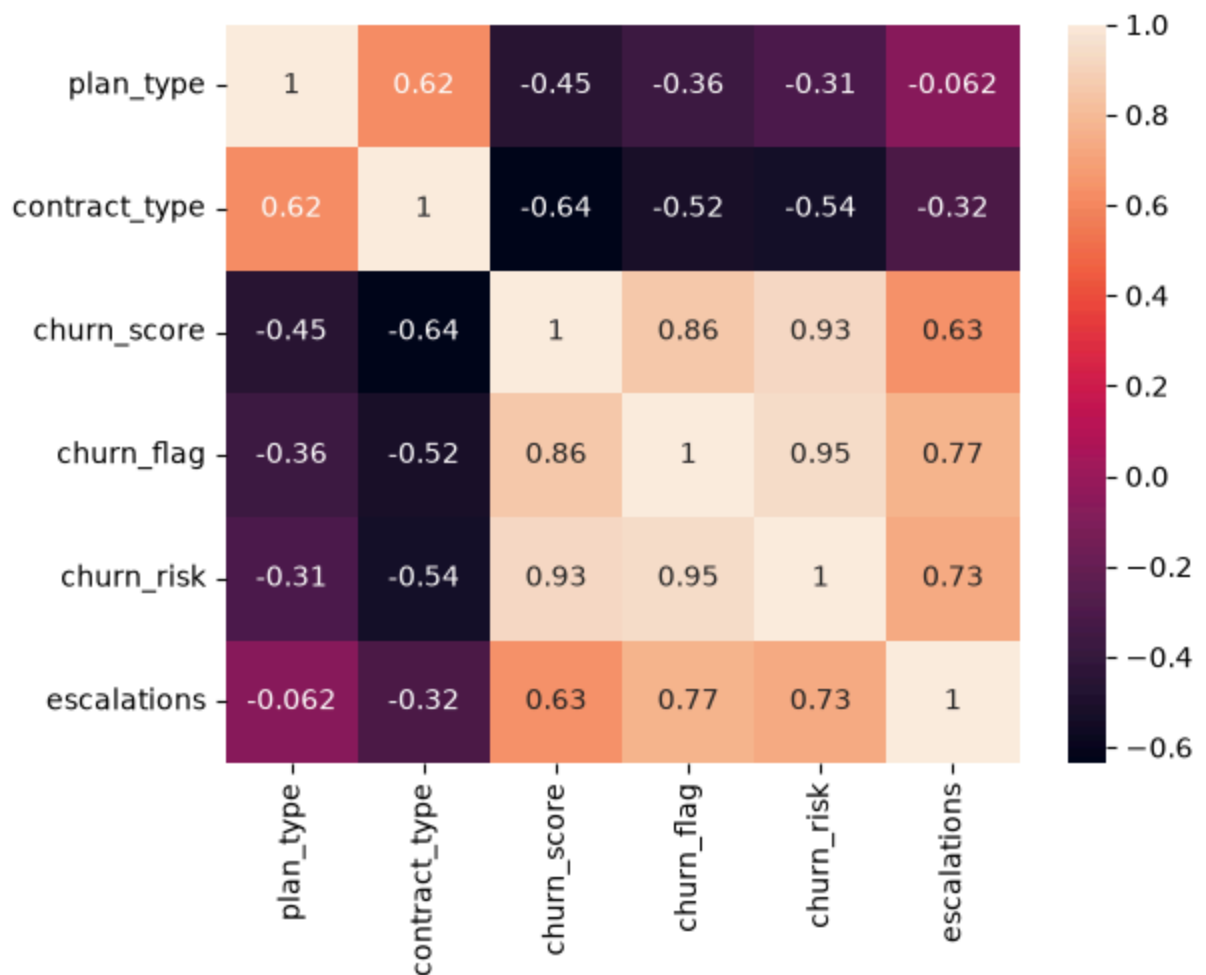
```

```
Out[74]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th>
<th>churn_flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>
```

```
1 1 12 0 0 0
2 1 91 1 2 1
0 0 34 0 0 0
2 1 8 0 0 0
1 0 88 1 2 1
```

```
In [75]: # Heatmap (correlation matrix)
sns.heatmap(df_encoded.corr(), annot=True)
```

```
Out[75]: <Axes: >
```



```
In [76]: # Heatmap Using Matplotlib
corr_matrix = df_encoded.corr() # Correlation matrix
```

```

fig, ax = plt.subplots(figsize=(10, 6)) # Create figure

cax = ax.imshow(corr_matrix, cmap='coolwarm') # Heatmap

fig.colorbar(cax) # Add colorbar

# Axis labels
ax.set_xticks(np.arange(len(corr_matrix.columns)))
ax.set_yticks(np.arange(len(corr_matrix.columns)))

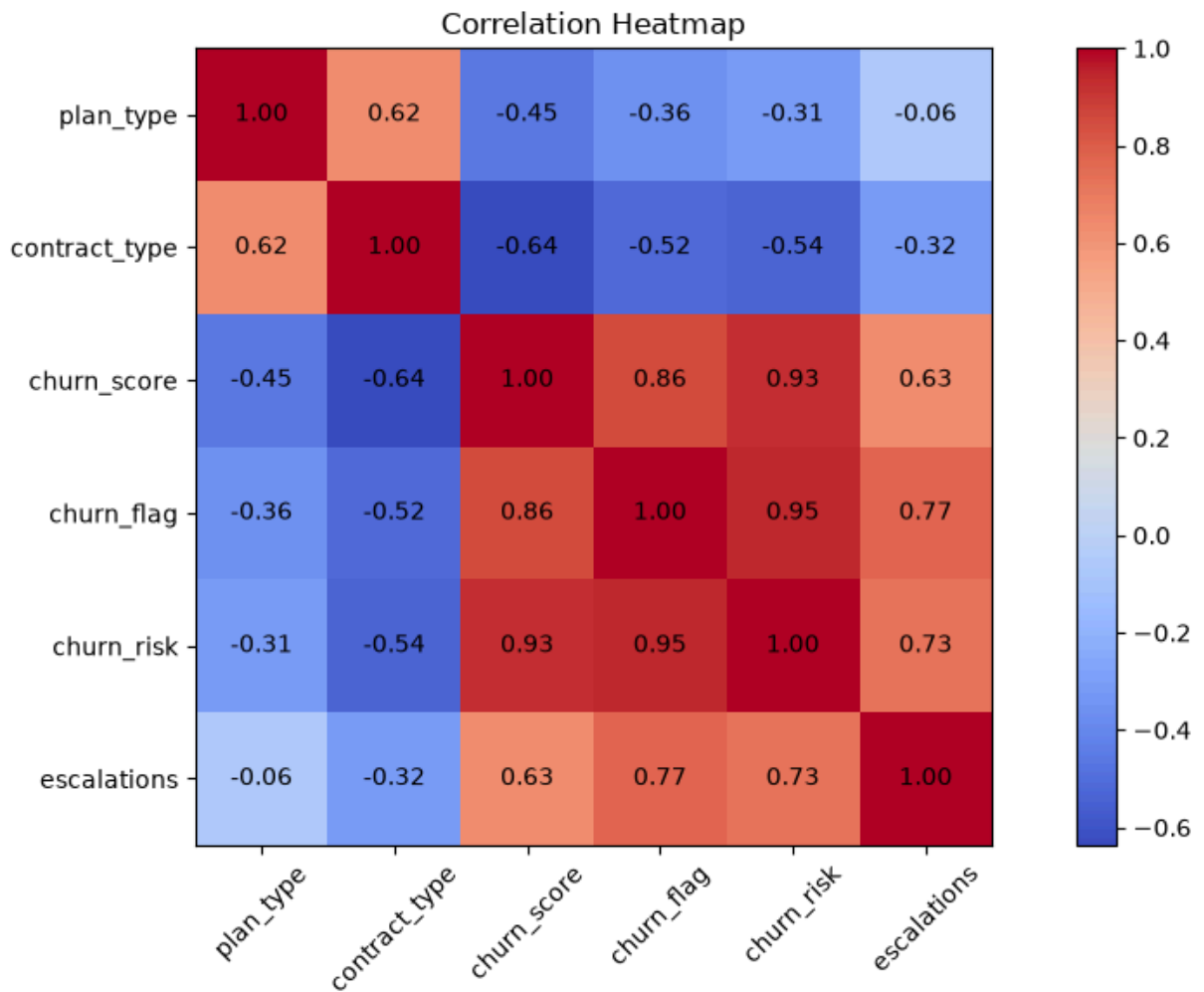
ax.set_xticklabels(corr_matrix.columns, rotation=45)
ax.set_yticklabels(corr_matrix.columns)

# Annotate values inside cells
for i in range(len(corr_matrix.columns)):
    for j in range(len(corr_matrix.columns)):
        ax.text(
            j,
            i,
            f"{corr_matrix.iloc[i, j]:.2f}",
            ha='center',
            va='center'
        )

plt.title('Correlation Heatmap') # Title

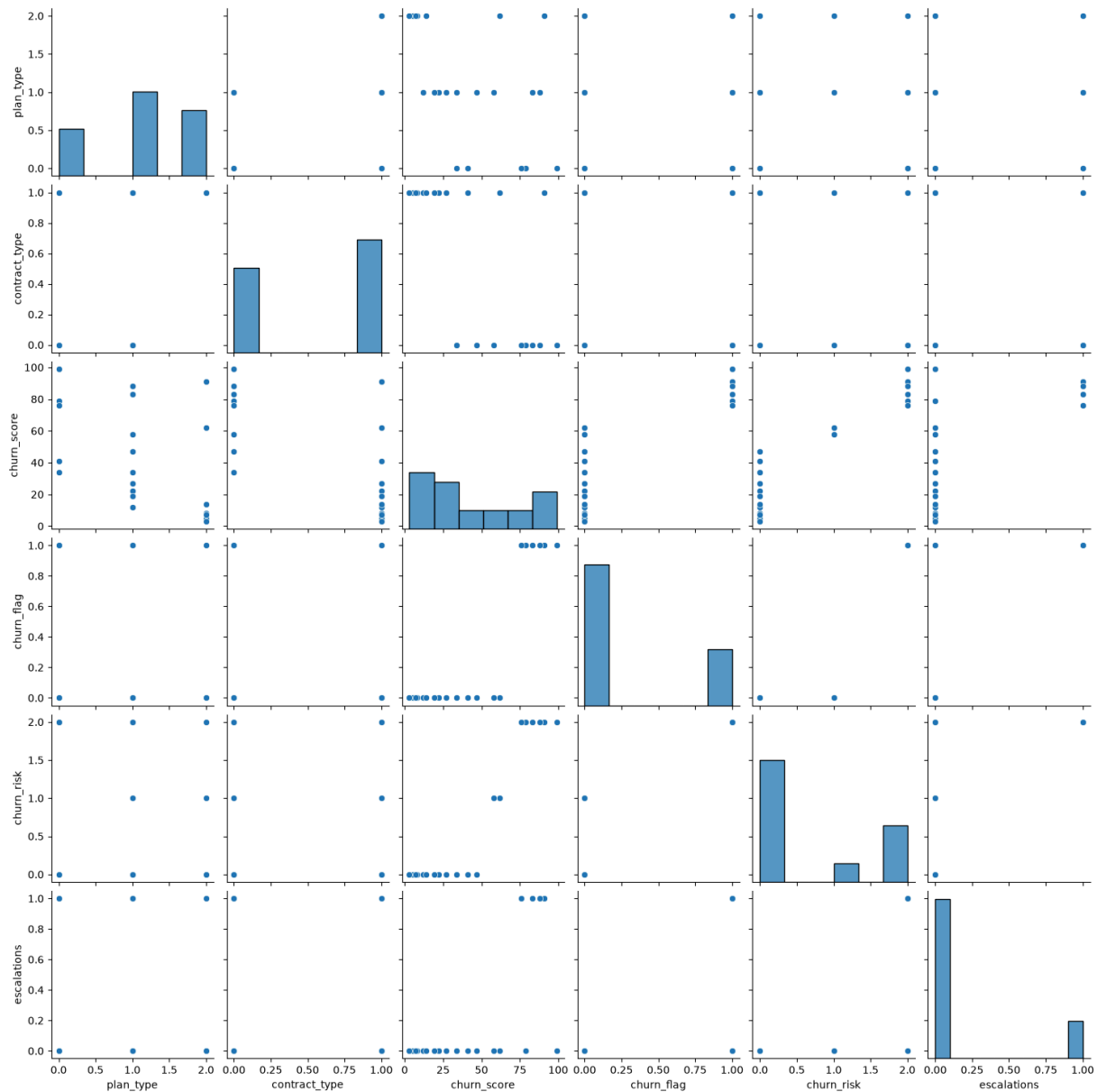
plt.tight_layout()
plt.show()

```



```
In [77]: # pairplot - Plot pairwise relationships in a dataset  
sns.pairplot(df_encoded)
```

```
Out[77]: <seaborn.axisgrid.PairGrid at 0x1c87f849010>
```



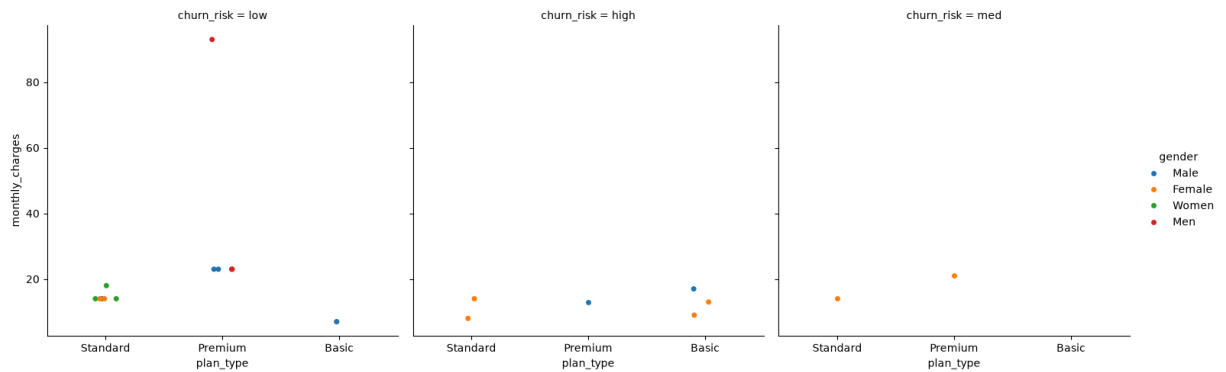
```
In [78]: df_visual.columns
```

```
Out[78]: Index(['customerid', 'subscription_start_date', 'subscription_type',
               'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
               'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'name',
               'country', 'state', 'gender', 'dob', 'interests', 'pincode',
               'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment',
               'complaint_count', 'churn_flag', 'churn', 'tenure_days', 'churn_risk',
               'cancellation_month'],
              dtype='str')
```

```
In [79]: # catplt/Facegrid plot - multi-dim comparison
```

```
sns.catplot(data=df_visual,
            x='plan_type',
            y='monthly_charges',
            hue='gender',
            col='churn_risk')
```

Out[79]: <seaborn.axisgrid.FacetGrid at 0x1c86ba60ad0>



In [80]: *# Pivot Table*

```
pd.pivot_table(  
    df_visual,  
    index='plan_type',  
    values='churn_flag',  
    aggfunc = 'mean'  
)
```

Out[80]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>churn_flag</th> <th>plan_type</th> <th></th> </thead> <tbody>
<th>Basic</th> <th>Premium</th> <th>Standard</th> </tbody> <tbody></tbody>

0.600000
0.142857
0.222222

In [81]: *# pivot table using multiple cols and agg type*

```
pd.pivot_table(  
    df_visual,  
    index='plan_type',  
    values=['monthly_charges', 'customerid', 'churn_flag'],  
    aggfunc = {  
        'monthly_charges' : 'sum',  
        'customerid' : 'nunique',  
        'churn_flag' : 'mean'  
    }  
)
```

```
Out[81]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>churn_flag</th> <th>customerid</th> <th>monthly_charges</th>
<th>plan_type</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>Basic</th>
<th>Premium</th> <th>Standard</th> </tbody> <tbody></tbody>

0.600000  5    52.95
0.142857  7   218.93
0.222222  9   123.91
```

In []:

In []: *# WORKING WITH SQL IN PYTHON*

```
In [82]: # create db in sql
conn = sqlite3.connect('test_database.sqlite')

#table details
conn.execute("CREATE TABLE users (first_name TEXT, country TEXT, budget INTEGER)")

# commit and save
conn.commit()

print("Cretaed Table successfully!")
```

Cretaed Table successfully!

```
In [83]: # insert data

cursor = conn.cursor()

# write sql query to insert records in sql table
cursor.execute(
    """
    INSERT INTO users VALUES
        ('Madhav', 'India', 5000),
        ('Rishabh', 'Germany', 2500),
        ('Vishakha', 'India', 3500)
    """
)

# commit and save
conn.commit()

print("Data Inserted successfully!")
```

Data Inserted successfully!

```
In [84]: # check inserted data in table
conn = sqlite3.connect('test_database.sqlite')
query = """SELECT * FROM users"""

df_results = pd.read_sql(query, conn)
```

```
df_results.head()
```

Out[84]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>first_name</th> <th>country</th> <th>budget</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

Madhav	India	5000
--------	-------	------

Rishabh	Germany	2500
---------	---------	------

Vishakha	India	3500
----------	-------	------

```
In [85]: # aggregation
query = """
        SELECT country, SUM(budget) as total_budget
        FROM users
        GROUP BY country
        """
df_agg = pd.read_sql(query, conn)
df_agg
```

Out[85]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>country</th> <th>total_budget</th> </thead> <tbody> <th>0</th>
<th>1</th> </tbody> <tbody></tbody>

Germany	2500
---------	------

India	8500
-------	------

```
In [86]: conn.close()
```

```
In [ ]: # PROJECT COMPLETE
```